Plan integrado por integrante

TPI · Bases de Datos 2 | Matías Yacob, Javier y Brenda
Guía v2: responsabilidades, contratos, ejemplos y criterios para integrar. Reemplaza las guías organizativas anteriores. Los ejemplos orientan la implementación; no representan código ya probado.

1. Objetivo común y límites

Aplicación de tienda con clientes, productos y pedidos. React → API Express → MongoDB Atlas. Respaldo externo al proceso web. Usar datos ficticios; sin pagos, envíos, stock ni login en esta demostración. Acordar autenticación antes de publicar escrituras en Internet.

Responsable	Parte propia	Entrega a los demás
Matías Yacob	Modelo, Atlas, API y reporte.	Contrato de API, datos semilla, backend probado y acceso de respaldo para Brenda por canal privado.
Javier	Frontend y conexión con API.	Pantallas, mocks compatibles, manejo de errores e instrucciones de ejecución.
Brenda	Backup, programación y restauración.	Scripts, pruebas y evidencia real; procedimiento de recuperación y RTO medido.

2. Repositorio y fronteras de trabajo

```
tpi-bases-datos-2/
  backend/      # Matias
  frontend/     # Javier
  scripts/      # Brenda
  docs/
    contrato-api.md
    modelo.md
    backend.md
    frontend.md
    respaldo.md
    pruebas-integracion.md
  resguardos_tpi/  # local; fuera de Git
```

Cada integrante trabaja en una rama propia y su carpeta. Matías mantiene contrato-api.md y modelo.md; los otros documentan su módulo. Versionar package-lock.json de cada app. No subir node_modules, .env, secretos, volcados ni archivos de configuración con credenciales.

3. Convenciones que todos respetan

Base: tpi_tienda. Colecciones: clientes, productos, pedidos. ObjectId en MongoDB, id como string en API; fechas BSON Date en base e ISO 8601 UTC en JSON. Importes enteros en centavos; 150050 equivale a $1500,50. Validar enteros seguros y cálculos dentro del rango seguro de JavaScript.

Nombres de campos y mensajes API sin acentos. Código comentado, separadores #==========# adaptados al lenguaje. Cambios de contrato se documentan y comunican antes de integrar. Ningún integrante modifica en silencio campos que consume otro.

Matías · modelo y Atlas

Herramientas: MongoDB Atlas, Compass, Node.js, Express, driver oficial MongoDB y dotenv. Elegir una versión LTS de Node compatible con ambas apps y registrar su versión exacta al iniciar.

4. Estructuras que debe implementar

Ejemplos esquemáticos: ObjectId y Date indican tipos BSON, no strings literales que deban persistirse.

```
// clientes
{
  _id: ObjectId, nombre: "Matias", apellido: "Yacob",
  email: "matias@example.com",
  contacto: { telefono: "1112345678", direccion: "Calle 123" },
  activo: true, createdAt: Date, updatedAt: Date, deletedAt: null
}
// productos
{
  _id: ObjectId, nombre: "Mouse", descripcion: "Mouse USB",
  categoria: "tecnologia", precioCentavos: 150000,
  activo: true, createdAt: Date, updatedAt: Date, deletedAt: null
}
```

Cliente: nombre, apellido y email obligatorios. Normalizar email a minúsculas y sin espacios exteriores; único, incluso en inactivos. Contacto y sus campos son opcionales; si faltan, devolver contacto={}. PATCH contacto reemplaza el objeto completo; Javier envía todos sus campos actuales.

Producto: nombre, categoria y precioCentavos obligatorios; descripcion opcional, por defecto cadena vacía. Categorías permitidas: tecnologia, libreria, hogar. Precio entero positivo. No incluir stock.

```
// pedidos
{
  _id: ObjectId, clienteId: ObjectId,
  cliente: { nombre: "Matias", apellido: "Yacob",
             email: "matias@example.com" },
  estado: "pendiente",
  items: [{ productoId: ObjectId, nombre: "Mouse",
    categoria: "tecnologia", precioUnitarioCentavos: 150000,
    cantidad: 2, subtotalCentavos: 300000 }],
  totalCentavos: 300000,
  activo: true, createdAt: Date, updatedAt: Date, deletedAt: null
}
```

Embedding: contacto e ítems se leen con su documento. Copias históricas de cliente y producto conservan el estado al comprar. **Linking:** clienteId y productoId apuntan a entidades compartidas. Cambios y bajas posteriores no alteran el historial.

5. Pasos de Atlas

Crear clúster gratuito, usuarios separados para app y respaldo, y acceso de red para los equipos autorizados. Crear validadores $jsonSchema, índices y datos semilla. Probar conexión. Entregar a Brenda nombre de base, host e instrucciones de acceso por canal privado; nunca publicar contraseñas.

Matías · backend y validación

6. Organización sugerida y configuración

```
backend/
  package.json       # scripts: dev, start, seed
  package-lock.json
  .env.example       # valores ilustrativos, sin secretos reales
  src/
    app.js           # Express, CORS, rutas y errores
    server.js        # conectar y escuchar
    config/db.js     # cliente MongoDB reutilizable
    routes/          # clientes, productos, pedidos, reportes
    services/        # validaciones y reglas de negocio
    repositories/    # consultas; filtros activo:true
    scripts/seed.js  # datos ficticios reproducibles

# .env.example del backend
PORT=3000
MONGODB_URI=<configurar-en-archivo-local>
MONGODB_DB=tpi_tienda
FRONTEND_ORIGIN=http://localhost:5173
```

Instalar dependencias y ejecutar npm run dev desde backend. El script seed no borra datos por defecto; documentar su ejecución y prevenir duplicados al repetirlo. Proponer 3 clientes, 9 productos y 6 pedidos, con estados variados y entidades inactivas de prueba.

7. CRUD y pedidos

CREATE establece activo=true, deletedAt=null y fechas del servidor. READ filtra activo=true. PATCH acepta solo campos permitidos y actualiza updatedAt. DELETE usa $set para activo=false, deletedAt y updatedAt; jamás deleteOne/deleteMany. Ausente o inactivo: 404.

PATCH cliente permite nombre, apellido, email, contacto; producto permite nombre, descripcion, categoria, precioCentavos; pedido solo estado. Rechazar campos desconocidos o reservados con 400. Rechazar identificadores mal formados con 400.

POST pedido: validar cliente y productos activos, entre 1 y 50 productos distintos y cantidades enteras de 1 a 100. Rechazar repetidos y campos de precio/total. Leer datos de MongoDB, calcular instantáneas, subtotales y total e insertar como pendiente. No modificar cliente ni ítems después de crear.

Estado	Puede pasar a
pendiente	confirmado o cancelado
confirmado	completado o cancelado
completado / cancelado	Ningún estado nuevo

Para evitar sobrescribir un cambio concurrente, actualizar usando el estado actual en el filtro. Una transición inválida o repetir el mismo estado devuelve 409. Cancelar y desactivar son operaciones distintas; no hay restauración de bajas en esta versión.

8. Entrega verificable de Matías

Backend que inicia, contrato actualizado, modelo y validadores, seed, pruebas y ejemplos. Demostrar: rechazo de referencias inactivas y email duplicado, total controlado por servidor, historial estable y filtros de baja lógica. Dar a Javier ids reales desde GET, no depender de ids fijos del seed.

Contrato para Matías y Javier

9. Rutas y filtros compartidos

Base local: http://localhost:3000/api. GET /health devuelve 200 si backend y base están disponibles; 503 ante conexión no disponible, sin datos sensibles.

Método	Ruta bajo /api	Uso
GET	/clientes, /productos, /pedidos	Listados activos.
GET	/clientes/:id, /productos/:id, /pedidos/:id	Detalle activo.
POST	/clientes, /productos, /pedidos	Crear.
PATCH	/clientes/:id, /productos/:id	Editar campos permitidos.
PATCH	/pedidos/:id	Cambiar estado.
DELETE	/clientes/:id, /productos/:id, /pedidos/:id	Baja lógica.
GET	/reportes/ventas-por-categoria	Reporte de completados.

Productos filtra categoria. Pedidos filtra clienteId, estado, desde, hasta. Listados admiten pagina (1 por defecto) y limite (10 por defecto, máximo 100), ambos enteros positivos. Orden createdAt descendente y _id descendente. total es el número de documentos coincidentes, no páginas.

desde/hasta usan ISO 8601 con zona y filtran createdAt; desde inclusivo, hasta exclusivo. Para un día completo, usar medianoche local y medianoche del siguiente día convertidas a ISO con zona. Rechazar intervalos invertidos. Filtros omitidos no restringen.

10. Ejemplos de solicitudes y respuestas

```
// POST /clientes
{ "nombre":"Matias", "apellido":"Yacob",
  "email":"matias@example.com", "contacto":{} }
// POST /productos
{ "nombre":"Mouse", "categoria":"tecnologia",
  "descripcion":"Mouse USB", "precioCentavos":150000 }
// POST /pedidos (ids ObjectId validos obtenidos de GET)
{ "clienteId":"507f1f77bcf86cd799439011", "items":[
  { "productoId":"507f191e810c19729de860ea", "cantidad":2 }
] }
// PATCH /pedidos/:id
{ "estado":"confirmado" }
// Lista vacia
{ "data":[], "paginacion":{"pagina":1,"limite":10,"total":0} }
// Error (campo es opcional)
{ "error":{"codigo":"VALIDACION", "mensaje":"Cantidad invalida",
            "campo":"items[0].cantidad"} }
```

Respuesta individual: {data: recurso completo}, con id string en lugar de _id, fechas ISO y referencias string, también dentro de ítems. DELETE: {data: {id, activo:false}}. HTTP: 200 lectura/cambio; 201 creación; 400 validación; 404 inexistente/inactivo; 409 duplicado/transición; 500 interno.

Códigos error estables: VALIDACION, NO_ENCONTRADO, EMAIL_DUPLICADO, TRANSICION_INVALIDA, ERROR_INTERNO. El frontend puede mostrar mensaje sin interpretar texto. Errores internos no exponen URI, stack ni credenciales.

Javier · frontend autónomo

11. Estructura y arranque

```
frontend/
  package.json           # scripts: dev, build, preview
  package-lock.json
  .env.example           # VITE_API_URL=http://localhost:3000/api
  src/
    App.jsx
    api/client.js        # fetch y manejo uniforme de errores
    pages/               # Clientes, Productos, Pedidos, Reporte
    components/          # formularios, tablas, mensajes
    mocks/               # datos y adaptador temporal
    styles/
```

React + Vite, HTML/CSS y fetch. Instalar dependencias y ejecutar npm run dev; fijar puerto 5173 para que coincida con CORS. Mantener mocks detrás del mismo adaptador que luego llama a la API; no duplicar reglas en cada pantalla.

12. Ejemplo de mock compatible

```
{
  "data":[{
    "id":"507f191e810c19729de860ea", "nombre":"Mouse",
    "descripcion":"Mouse USB", "categoria":"tecnologia",
    "precioCentavos":150000, "activo":true,
    "createdAt":"2026-10-15T12:00:00.000Z",
    "updatedAt":"2026-10-15T12:00:00.000Z", "deletedAt":null
  }],
  "paginacion":{"pagina":1,"limite":10,"total":1}
}
```

13. Pantallas y comportamiento

Clientes: listado paginado, alta, edición y confirmación de baja. **Productos:** igual, más filtro por categoría. **Pedidos:** elegir cliente y productos activos, cantidades, detalle histórico, cambios de estado válidos y baja separada de cancelación. **Reporte:** fechas, unidades e importes por categoría.

Cada vista contempla carga, lista vacía, éxito y error. Tras un cambio exitoso, refrescar la consulta y conservar filtros cuando corresponda. Leer response.ok; errores HTTP no son lo mismo que un fallo de red. Mostrar el mensaje del backend y asociar campo al formulario si existe.

Mostrar precioCentavos/100 como moneda ARS. Al editar precios, convertir el texto decimal a centavos enteros validando dos decimales como máximo. Al crear pedido enviar solo clienteId y productoId/cantidad. Un total visual es estimativo; tras guardar mostrar el total del servidor.

No obtener nombres históricos del catálogo actual: usar cliente e items del pedido. No enviar activo, fechas o ids en altas. No colocar secretos en variables VITE_: son públicas. No enviar solicitudes a Atlas desde el navegador.

14. Entrega verificable de Javier

Mock funcional sin backend, build exitoso, instrucciones y capturas reales. Luego integrar URL de API, sustituir mocks, probar 400/404/409 y flujos completos. Registrar lo que necesita de Matías: API disponible y datos de prueba, no acceso directo a credenciales.

Matías · reporte e índices

15. Contrato del reporte que consume Javier

GET /api/reportes/ventas-por-categoria?desde=...&hasta=... usa createdAt del pedido. Considerar solo activo=true y estado=completado. Agrupar por items.categoria histórica, sumar items.cantidad e items.subtotalCentavos y ordenar categoria ascendente. Sin datos: {data:[]}.

```
{
  "data":[{
    "categoria":"tecnologia", "unidadesVendidas":2,
    "importeCentavos":300000
  }]
}
```

Pipeline sugerido: $match → $unwind de items → $group → $project con los nombres del contrato → $sort. No sumar totalCentavos después de unwind: se duplicaría por cada ítem.

16. Índices iniciales y evidencia

Colección	Índice	Motivo
clientes	{email:1}, unique:true	Unicidad permanente.
productos	{activo:1,categoria:1,createdAt:-1,_id:-1}	Catálogo por categoría.
pedidos	{activo:1,createdAt:-1,_id:-1}	Listado general.
pedidos	{clienteId:1,activo:1,estado:1,createdAt:-1,_id:-1}	Cliente y estado.

Comprobar consultas reales con explain("executionStats"). Comparar planes y documentos examinados antes/después; no afirmar que un índice mejora todo. Ajustar según filtros reales. Validadores de MongoDB comprueban estructura; existencia/estado de referencias y transiciones corresponden al backend.

17. Casos que Matías comparte con Javier y Brenda

Caso	Resultado esperado
Cambiar precio del Mouse después de crear un pedido	El precio y total históricos del pedido no cambian.
Desactivar cliente/producto	No aparece en GET normal; pedidos previos se mantienen.
Crear pedido con referencia inactiva	404 y ningún pedido nuevo.
Enviar total o precio en POST pedido	400; no confiar en datos del navegador.
Repetir email con distinta capitalización	409 luego de normalizar.
Completar un pedido confirmado	Aparece en reporte si está activo y dentro de fechas.

Brenda usa estas consultas sobre una restauración para demostrar que los datos recuperados sirven, no solo que mongorestore termina sin error. Matías proporciona un resumen de conteos de las tres colecciones previo al respaldo para contrastar.

Brenda · respaldo autónomo

18. Estructura, entradas y dependencia mínima

```
scripts/
  backup.ps1
  programar-backup.ps1
  restaurar-prueba.ps1
  backup.config.example.yml  # solo plantilla sin secretos
docs/respaldo.md
resguardos_tpi/
  2026-10-15/
    tpi_tienda/
      clientes.bson
      clientes.metadata.json
      productos.bson
      productos.metadata.json
      pedidos.bson
      pedidos.metadata.json
```

La configuración real vive fuera de Git con permisos de archivo restrictivos. Usar el mecanismo de configuración protegido de Database Tools y consultar su sintaxis instalada. No poner URI con contraseña en argumentos visibles ni logs. Matías entrega host, base y cuenta por canal privado; autoriza la IP del equipo de Brenda.

Resolver la raíz desde $PSScriptRoot, no desde el directorio actual. Rutas de respaldo relativas a esa raíz. Definir rutas de herramientas si no están en PATH. El script debe funcionar aunque Express esté detenido.

19. Interfaz de scripts acordada

```
# Desde la raiz del repositorio, en PowerShell
./scripts/backup.ps1
./scripts/backup.ps1 -Forzar
./scripts/programar-backup.ps1 -Hora "02:00"
./scripts/restaurar-prueba.ps1 -Fecha "2026-10-15"
# Destino fijo de restauracion: tpi_tienda_restore_test
```

Los parámetros anteriores son el contrato a implementar. Todos los scripts devuelven 0 al terminar correctamente y un código distinto de cero ante error. Registrar inicio, fin, ruta final y resultado sin secretos. Una omisión por copia válida del día puede terminar con 0 y un mensaje claro.

20. Algoritmo del respaldo

1. Validar herramientas, configuración, acceso y raíz. Evitar dos ejecuciones simultáneas mediante bloqueo. 2. Crear carpeta temporal única bajo resguardos_tpi. 3. Ejecutar mongodump solo para tpi_tienda. 4. Validar código y BSON/metadatos de las colecciones esperadas. 5. Publicar carpeta AAAA-MM-DD. 6. Solo después, conservar las cinco fechas completas más recientes.

Si falla cualquier paso, conservar copias anteriores y reportar el error. Nunca borrar la única copia válida antes de tener una nueva. -Forzar prepara y valida otra copia antes de sustituir la del día. Borrar únicamente carpetas verificadas dentro de resguardos_tpi, con formato fecha y evidencia de copia completa.

Brenda · recuperación y pruebas

21. Programación diaria

La tarea diaria se llama TPI-Tienda-Backup y no debe duplicarse al registrar de nuevo. Lanzar PowerShell sin interacción usando rutas absolutas al script; el script resuelve sus propias rutas relativas. Elegir una cuenta con acceso a secretos, herramientas y disco. Configurar recuperación de ejecuciones perdidas y evitar instancias superpuestas.

El equipo debe estar encendido, con red, IP autorizada y disco persistente. Aclarar estas condiciones al grupo: una app publicada no ejecuta por sí sola la tarea Windows. No ofrecer respaldo diario como probado antes de verificar la tarea registrada y su ejecución.

22. Restauración aislada

Elegir una copia válida y restaurar solo en tpi_tienda_restore_test, nunca en tpi_tienda. Validar destino antes de ejecutar; si la base de prueba tiene datos, pedir confirmación explícita antes de reemplazarlos. Documentar mapeo de namespace origen/destino de mongorestore.

Medir desde inicio de recuperación hasta que las consultas respondan correctamente. Comprobar conteos de clientes/productos/pedidos, un pedido con sus ítems, baja lógica y reporte. Usar cuenta con permisos de restauración solo en el destino necesario.

Prueba	Evidencia a registrar
Respaldo manual correcto	Fecha, código de salida, carpetas y archivos sin secretos.
Fallo de conexión	Error informado; respaldos anteriores intactos.
Más de cinco copias completas	Se retienen las cinco más recientes; no se toca otro directorio.
Copia existente y ejecución forzada	Omisión normal y sustitución segura al forzar.
Tarea diaria y app detenida	Configuración de tarea y resultado real de ejecución.
Restauración de prueba	Destino aislado, conteos, consultas y duración medida.

23. Entrega verificable de Brenda

Scripts, plantilla sin secretos, documentación de instalación/ejecución, logs sanitizados, capturas y prueba de restauración. RPO estimado hasta unas 24 horas si las tareas se completan; aumenta ante fallos o equipo apagado. RTO medido, no inventado.

mongodump genera un volcado lógico BSON, no una instantánea física. Para una prueba coherente del TPI, pausar escrituras durante el respaldo y registrar esa condición; no afirmar que es una copia transaccional global bajo escrituras concurrentes.

Integración de las tres partes

24. Orden de unión y verificaciones

Etapa	Acción	Criterio para avanzar
1. Contrato	Leer esta guía y guardar ejemplos en docs/contrato-api.md.	Todos usan los mismos nombres, precios y estados.
2. Desarrollo paralelo	Matías API; Javier mocks; Brenda herramientas/scripts.	Cada módulo inicia o ejecuta de forma independiente.
3. Front + API	Cambiar adaptador mock por fetch al backend.	CRUD, filtros, paginación y errores funcionan.
4. Datos + backup	Ejecutar backup con datos conocidos y sin escrituras.	Copia validada; cinco copias; fallo seguro.
5. Recuperación	Restaurar en base de prueba y consultar.	Conteos y casos de negocio correctos; RTO medido.
6. Entrega	Informe y ZIP final sin secretos/dependencias.	Los tres pueden explicar y ejecutar todo.

25. Documentación obligatoria de cada módulo

Cada ficha incluye objetivo, responsable/revisor, entradas y salidas, archivos, decisiones, requisitos, comandos de arranque, resultado esperado, resultado real, evidencias, problemas y solución. Incluir preguntas de defensa y dependencias con otras partes.

```
# Ejemplo de ficha: alta de pedido
Responsable: Matias | Revisor: Javier
Entrada: clienteId y lista de productoId/cantidad
Salida: HTTP 201, pedido con instantaneas y total calculado
Prueba: alterar precio en la solicitud -> HTTP 400
Evidencia: solicitud y respuesta sin secretos
Defensa: por que los items estan embebidos y tienen productoId
```

26. Revisión cruzada y cierre

Matías revisa a Brenda; Brenda revisa a Javier; Javier revisa a Matías. El revisor ejecuta los pasos sin depender de explicaciones improvisadas. Registrar dudas y actualizar documentación. Cada integrante ensaya también la explicación de una parte ajena.

Informe PDF: modelo, embedding/linking, Atlas, CRUD/baja lógica, índices, pipeline, respaldo, restauración, RPO/RTO y conclusiones con evidencia real. ZIP: código, scripts y documentación; excluir node_modules, archivos .env reales, contraseñas y respaldos. Esta entrega de BD2 no sigue la regla de solo src de los trabajos Java.

Control de cambios: rutas, campos, mensajes/códigos y parámetros de scripts son contratos. Si hay que cambiarlos, registrar qué cambia, quién consume el cambio y qué prueba lo verifica; avisar antes de integrar. Confirmar fechas y requisitos formales con el campus.